



Prompt Engineering & Tool Calling

AICB-P1 · Ngày 4 · Làm sao nói để AI hiểu đúng ý?

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

*“Hai người hỏi AI cùng một việc,
một người nhận kết quả xuất sắc,
người kia nhận rác. Tại sao?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Prompt fundamentals
2. Advanced prompting techniques
3. System prompt engineering
4. Context engineering
5. Tool calling
6. Design principles cho tools
7. Parallel tool calls & patterns
8. Lab 4 + deliverable cuối buổi

- Viết được prompt rõ ràng theo các thành phần **Role / Task / Context / Format**
- Hiểu khi nào nên dùng **zero-shot, few-shot, CoT**, và khi nào không cần
- Viết được **system prompt production-grade** cho agent
- Khai báo được **tool schema** và hiểu vòng lặp tool calling từ model đến tool rồi quay lại model

Mục tiêu của buổi này là hiểu **cơ chế**: prompt là interface giữa human intent và model behavior; tool calling là interface giữa model và thế giới bên ngoài.

1 agent script chạy được + 1 system prompt + 2 tool schemas + 5 test questions
+ ghi chú lỗi prompt/tool/control flow

- 2 tools tự viết: 1 API wrapper đơn giản, 1 data query đơn giản
- 1 system prompt có rules, constraints, output contract
- 5 câu test để chứng minh agent biết khi nào trả lời trực tiếp, khi nào gọi tool

Prompt Engineering Funda- ~~mentals~~

Prompt tốt không phải prompt “hay”, mà là prompt tạo ra hành vi mong muốn ổn định

Prompt kém

“Viết email cho tôi”

Không rõ gửi ai, về gì, tone nào, dài bao nhiêu.

Kết quả: chung chung, khó dùng ngay.

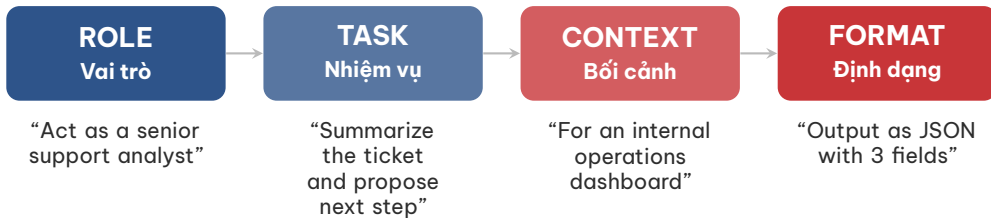
Prompt tốt

Viết email xin lỗi khách hàng
về giao hàng trễ 2 ngày,
tone lịch sự, dưới 120 từ,
có CTA rõ ràng.

Rõ task, context, constraint, format.

Kết quả: actionable hơn hẳn.

Lưu ý: Nguyên tắc vàng: **Specificity beats cleverness**. Prompt ngắn nhưng rõ nghĩa thường tốt hơn prompt dài mà lan man.



Bắt đầu với **Task + Format**. Chỉ thêm Role hoặc Context khi chúng thực sự cải thiện chất lượng hoặc tính nhất quán.

Loại prompt	Mục đích chính	Khi dùng
Instruction prompt	Ra lệnh trực tiếp cho một tác vụ	Hỏi đáp 1 lượt, transform, summarize, classify
Conversation prompt	Giữ ngữ cảnh nhiều lượt với user	Chatbot, support, tutor, debugging nhiều bước
System prompt	Đặt policy, boundary, output contract	Agent, assistant production, use case cần hành vi ổn định

Anthropic prompting guidance + teaching heuristics

- Prompt dài hơn **không đồng nghĩa** prompt tốt hơn.
- Mỗi token thừa làm tăng **chi phí, latency**, và đôi khi cả nhiễu.
- Hãy ưu tiên: instruction rõ, examples đúng chỗ, output contract rõ.
- Rule thực dụng: nếu prompt dài thêm nhưng không làm thay đổi hành vi mong muốn, hãy cắt bớt.

Lưu ý: Prompt engineering tốt là tối ưu **độ rõ** và **khả năng kiểm soát**, không phải thi xem ai viết prompt dài hơn.

Advanced Prompting Techniques

Dùng kỹ thuật nâng cao khi chúng cải thiện chất lượng thật sự, không dùng như thần chú

Zero-shot

Không có ví dụ mẫu.
Nhanh, rẻ, nên thử trước.

One-shot

1 ví dụ mẫu.
Tốt khi cần giữ format rõ hơn.

Few-shot

2-5 ví dụ.
Tăng consistency, nhưng tốn token hơn.

CoT

Cho model reasoning từng bước.
Hữu ích cho task suy luận.

Thứ tự thử thực dụng: **zero-shot** -> **few-shot** -> **decomposition / CoT**. Đừng nhảy vào prompt phức tạp ngay từ đầu.



- Khi model hiểu task nhưng **ra sai format** hoặc **không ổn định giữa các input tương tự**.
- Khi cần giữ tiêu chuẩn đánh giá, tone, hoặc cách lập luận nhất quán.
- Ví dụ mẫu nên **relevant**, **đa dạng vừa đủ**, và **đúng format mong muốn**.

Few-shot không phải để “dạy lại” model mọi thứ; nó là cách chỉ ra pattern mà bạn muốn model bám theo.

Nguồn minh họa: zero/few-shot teaching graphic trong repo

```
examples = """
Input: "Great product, fast delivery!"
Output: Positive

Input: "Terrible quality, waste of money"
Output: Negative
"""

prompt = f"""Classify feedback as Positive, Negative, or Neutral.

{examples}
Input: "Love the design but shipping was slow"
Output: ""
print(prompt)
```

CoT phù hợp khi:

- Bài toán cần reasoning nhiều bước
- Bạn muốn model giải thích logic trung gian
- Bạn cần debug xem model sai ở bước nào

Tree-of-Thought:

- Hữu ích cho bài toán cần explore nhiều hướng
- Phức tạp hơn, tốn token và latency hơn
- Chỉ nên giới thiệu như extension, không phải mặc định cho mọi task

CoT là công cụ cải thiện reasoning, không phải phép màu. Nếu task vốn dĩ chỉ là formatting hoặc extraction đơn giản, CoT thường là overkill.

System Prompt Engineering

System prompt tốt làm agent nhất quán hơn, dễ kiểm soát hơn, và dễ test hơn

Persona: role, expertise level, communication style

Rules: việc nên làm, việc luôn phải làm

Capabilities: model được phép dùng tools nào, dữ liệu nào

Constraints: không làm gì, khi nào từ chối, khi nào escalate

Output format: JSON, markdown, bullet list, schema, language

priority



```
system_prompt = """
You are a support triage agent for an e-commerce team.

Rules:
- Answer in Vietnamese.
- Be concise and operational.
- If billing or refund policy is unclear, ask for more details.

Constraints:
- Never invent order status.
- Never promise refunds without tool confirmation.

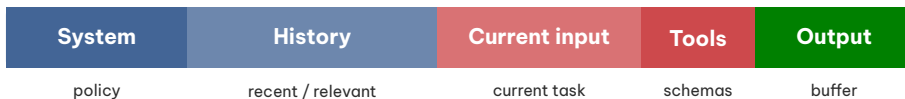
Output format:
Return JSON with: intent, action, reply
"""
```

- ☐ **Quá dài:** nhồi mọi thứ vào 1 prompt 2000+ tokens rồi hy vọng model luôn làm đúng
- ☐ **Mâu thuẫn:** vừa bảo “ngắn gọn”, vừa bắt “giải thích chi tiết từng bước”
- ☐ **Mơ hồ:** “hãy thông minh”, “hãy chuyên nghiệp”, nhưng không định nghĩa chuẩn output
- ☐ **Không test edge cases:** quên kiểm tra câu hỏi ngoài phạm vi, refusal, tool failure
- ☒ **Nguyên tắc:** system prompt là policy layer. Càng rõ boundary, càng dễ predict hành vi

Context Engineering

Điều quan trọng không phải nhét bao nhiêu context, mà là chọn đúng context cần thiết





Lưu ý: Token budget allocation cần chủ động: đừng để history, tools, và examples ăn hết chỗ dành cho output.

Memory injection

- Chỉ đưa vào facts thật sự cần cho task hiện tại
- Ưu tiên recent history hoặc relevant history, không dump toàn bộ transcript
- Tốt cho support agent, coding assistant, tutor nhiều lượt

Compression

- **Summarize:** tóm tắt phần cũ
- **Drop:** bỏ hẳn phần không còn liên quan
- **Archive:** đẩy ra ngoài context, chỉ fetch lại khi cần

Context engineering là bài toán chọn lọc và ưu tiên. Nếu mọi thứ đều quan trọng, thực ra không có gì thực sự nổi bật với model.

Rổ token	Chứa gì	Rủi ro nếu quá nhiều
System prompt	policy, rules, output format	chậm hơn, khó maintain
History	recent turns, facts liên quan	dễ nhiễu, dễ lost in the middle
Tool schemas	tên tool, mô tả, tham số	model chọn tool tệ nếu schema dài hoặc mơ hồ
Output buffer	phần model dùng để trả lời	bị cắt cụt output nếu cấp thiếu

Teaching heuristic for token budgeting

Tool Calling

Tool calling là cách agent chuyển từ “nói” sang “tương tác với thế giới thực”

05



Model không tự chạy code hay tự gọi API ngoài. Ứng dụng của bạn nhận tool request, chạy tool, rồi gửi kết quả trở lại model.

- **Name:** nên ngắn, rõ, động từ đúng việc
- **Description:** nói khi nào nên dùng tool này
- **Parameters:** mô tả input bằng JSON Schema
- **Required fields:** giúp model biết thiếu gì thì chưa gọi được

Lưu ý: LLM đọc description như tài liệu hướng dẫn. Nếu description mơ hồ, model sẽ chọn sai tool hoặc truyền sai arguments.

```
weather_tool = {  
    "type": "function",  
    "function": {  
        "name": "get_weather",  
        "description": "Get current weather for a city when the user asks about weather conditions.",  
        "parameters": {  
            "type": "object",  
            "properties": {  
                "city": {"type": "string", "description": "City name, e.g. Hanoi"}  
            },  
            "required": ["city"]  
        }  
    }  
}
```

Design Principles Cho Tools

Tool tốt là software interface tốt, không phải prompt trang trí

Nguyên tắc	Ý nghĩa	Nếu vi phạm
Single Responsibility	Mỗi tool làm 1 việc rõ ràng	model khó quyết định nên gọi tool nào
Idempotency	Cùng input cho cùng kết quả; side effect được kiểm soát	retry dễ sinh lỗi phụ
Granularity hợp lý	Không quá nhỏ, cũng không ôm quá nhiều việc	hoặc overhead lớn, hoặc tool quá cứng
Test độc lập	Unit test từng tool trước khi gắn vào agent	khó tách lỗi tool khỏi lỗi prompt

Principles for reliable tool interfaces

Quá nhỏ

- `get_customer_name`
- `get_customer_email`
- `get_customer_phone`

Hệ quả: quá nhiều calls, overhead lớn, flow rối.

Quá to

- `handle_all_customer_operations`

Hệ quả: model không hiểu boundary, khó debug, khó reuse.

Thiết kế tool quanh một hành động nghiệp vụ rõ ràng: ví dụ `lookup_order`, `get_weather`, `query_sales_data`, `send_email_draft`.

Parallel Tool Calling & Patterns

Nhanh hơn không có nghĩa là tốt hơn nếu flow control và merge logic không rõ

Sequential

Tool B cần output của Tool A.
Ví dụ: tìm order ID -> rồi mới tra shipping status.

Parallel

Các tool độc lập có thể chạy cùng lúc.
Ví dụ: gọi thời tiết, tỷ giá, và lịch học song song.

Lưu ý: Chỉ song song hóa khi không có phụ thuộc dữ liệu. Nếu vẫn cần bước merge / verify rõ ràng ở cuối.

1. **Conditional tool use:** agent tự quyết định có cần tool hay trả lời trực tiếp.
2. **Tool chaining:** output của tool A là input của tool B.
3. **Parallel fetch + merge:** lấy nhiều nguồn độc lập rồi tổng hợp kết quả.

Tool calling không chỉ là “gọi API”. Nó là bài toán control flow: khi nào gọi, gọi cái gì, gọi theo thứ tự nào, và làm gì khi tool fail.

```
messages = [{"role": "user", "content": "ỜThị ếtit Hà ộNi và ýt giá USD hôm nay?"}]
response = client.responses.create(model="gpt-4.1", input=messages, tools=tools)

for item in response.output:
    if item.type == "function_call":
        result = run_tool(item.name, json.loads(item.arguments))
        messages.append(item)
        messages.append({"type": "function_call_output", "call_id": item.call_id, "output": result})

final = client.responses.create(model="gpt-4.1", input=messages, tools=tools)
print(final.output_text)
```

Thực Hành

Lab 4: Build first agent với system prompt + 2 tools + 5
test cases

08

1. Viết 1 system prompt với rules, constraints, output format
2. Tạo 2 custom tools: 1 API wrapper đơn giản, 1 data query đơn giản
3. Nối tools vào agent loop
4. Chạy 5 câu test để xem khi nào agent trả lời trực tiếp, khi nào gọi tool
5. Ghi lại lỗi thuộc loại prompt, tool schema, hay control flow

```
SYSTEM_PROMPT = open("system_prompt.txt").read()
TOOLS = [get_weather_tool(), query_sales_tool()]

while True:
    user_input = input("You: ")
    messages.append({"role": "user", "content": user_input})
    response = call_model(messages, SYSTEM_PROMPT, TOOLS)
    messages = handle_tool_calls(response, messages)
    print(render_final_answer(messages, SYSTEM_PROMPT, TOOLS))
```

Mục tiêu: Build ReAct agent với 2 custom tools, viết system prompt chuẩn, và test end-to-end trên 5 câu hỏi

Deliverable: Agent script chạy được + system prompt + 2 tool schemas + 5 test outputs + note lỗi prompt/tool/control flow

Thời gian: 150 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Prompt = interface giữa human intent và model capability. Prompt tốt giúp model làm đúng việc, đúng format, đúng boundary.
- 2 System prompt tốt = agent nhất quán và predictable hơn, đặc biệt khi có tools và constraints.
- 3 Tool schema description quyết định rất mạnh việc model biết khi nào dùng tool nào và gọi với arguments gì.
- 4 Parallel tool calls nhanh hơn đáng kể khi các tool độc lập; nếu có phụ thuộc dữ liệu, hãy giữ flow tuần tự.

AI Product Thinking & Requirements

“Bạn đã build được agent đầu tiên. Nhưng build xong chưa đủ. Ngày mai: sản phẩm này dành cho ai, yêu cầu ra sao, và rủi ro nào phải nghĩ từ đầu?”

- Hoàn thiện Lab 4 với 5 test questions rõ pass/fail
- Đọc lại system prompt của mình và chỉ ra 2 chỗ còn mơ hồ hoặc mâu thuẫn

- 1 Anthropic. *Prompt Engineering Overview*. platform.claude.com/docs
- 2 Anthropic. *Claude Prompting Best Practices và Multishot Prompting*. platform.claude.com/docs
- 3 Anthropic. *Tool Use Overview*. platform.claude.com/docs
- 4 OpenAI. *Function Calling Guide*. developers.openai.com/api/docs/guides/function-calling
- 5 Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022.
- 6 LangGraph Docs. *Quickstart*. langchain-ai.github.io/langgraph

Hỏi & Đáp

*Bạn đang gặp lỗi vì model chưa hiểu ý bạn,
hay vì tool contract của bạn chưa đủ rõ?*



Cảm ơn!

Email: lecturer@vinuni.edu.vn

Slides & tài liệu: github.com/aicb-vinuni

Lab template: bit.ly/aicb-day04-lab